

07-Hugging-Face-Hub

1 Hugging Face Hub API

Document Version: 1.0
Generated: December 4, 2025
Standard: Hugging Face Model Hub Protocol
Status: Community Standard

1.1 Overview

Hugging Face Hub is a central repository for machine learning models, datasets, and spaces. The Hub API provides programmatic access for uploading, downloading, and managing models with integrated model cards, versioning, and community features.

1.1.1 Key Features

- **Model Repository:** Central hub for sharing models
- **Model Cards:** Integrated documentation
- **Multiple Backends:** Support for Transformers, Diffusers, etc.
- **Versioning:** Git-like version control
- **Access Control:** Public/private models
- **Community:** Social features and discussions
- **API Access:** REST and Python SDK

1.1.2 Primary Use Cases

1. **Model Sharing:** Distribute trained models
 2. **Model Discovery:** Find community models
 3. **Model Cards:** Standardized documentation
 4. **Fine-tuning:** Start from pre-trained models
 5. **Inference:** Direct model access via API
-

1.2 Authoritative References

- **Hugging Face Hub Docs:** <https://huggingface.co/docs/hub>
 - **Hub API Reference:** <https://huggingface.co/docs/hub/api>
 - **Model Cards:** <https://huggingface.co/docs/hub/models-cards>
 - **GitHub:** <https://github.com/huggingface/hub>
-

1.3 Format Structure

1.3.1 Model Repository Structure

```
model-repo/
├── README.md           # Model card
├── config.json         # Model configuration
├── model.safetensors   # Model weights (safetensors format)
├── tokenizer.json      # Tokenizer config
├── tokenizer.model     # Tokenizer vocab
├── special_tokens_map.json # Special tokens
├── preprocessor_config.json # Processor config
└── .gitattributes     # Git LFS pointers
```

1.3.2 Model Configuration (JSON)

Real example (config.json):

```
{
  "_name_or_path": "bert-base-uncased",
  "architectures": ["BertForSequenceClassification"],
  "attention_probs_dropout_prob": 0.1,
  "directionality": "bidi",
  "hidden_act": "gelu",
  "hidden_dropout_prob": 0.1,
  "hidden_size": 768,
  "initializer_range": 0.02,
  "intermediate_size": 3072,
  "layer_norm_eps": 1e-12,
  "max_position_embeddings": 512,
  "model_type": "bert",
  "num_attention_heads": 12,
  "num_hidden_layers": 12,
  "output_past": true,
  "pad_token_id": 0,
  "position_embedding_type": "absolute",
  "transformers_version": "4.30.0",
  "type_vocab_size": 2,
  "use_cache": true,
  "vocab_size": 30522
}
```

1.3.3 Model Card Format (Markdown)

Real example (README.md):

```
---
Language:
  - en
License: mit
Library_name: transformers
tags:
  - text-classification
  - bert
widget:
  - text: "This movie is absolutely fantastic!"
    example_title: "Positive example"
  - text: "Terrible movie, waste of time."
    example_title: "Negative example"
model-index:
  - name: Results
    results:
      - task:
          name: Text Classification
          type: text-classification
        dataset:
          name: IMDb
          type: imdb
        metrics:
          - name: Accuracy
            type: accuracy
            value: 0.947
---

# bert-imdb-classifier

This model is a BERT model fine-tuned on the IMDb dataset for sentiment classification.

## Model Details

### Model Description

- Developed by: Hugging Face
- Model type: Transformer
- Library: Transformers
```

```
- **License**: MIT

### Intended Use

Primary use cases for this model:
- Sentiment analysis on movie reviews
- Text classification tasks

### Out-of-Scope

- Medical/legal document analysis
- Languages other than English

## Training Data

Trained on the IMDb movie reviews dataset with 50,000 labeled examples.

## Results

| Metric | Value |
|-----|-----|
| Accuracy | 94.7% |
| F1 Score | 0.944 |
| AUC-ROC | 0.978 |

## How to Use

```python
from transformers import pipeline

classifier = pipeline("text-classification", model="username/bert-imdb-classifier")
result = classifier("This movie was amazing!")
print(result)
```

## 1.4 Limitations

- Trained only on English text
- Best performance on movie reviews (domain-specific)
- May not generalize well to other domains

## 1.5 Environmental Impact

Model inference generates approximately 0.012kg CO<sub>2</sub>.

## 1.6 Bias and Fairness

Tested on demographic groups with similar performance across genders and ages.

```

Procedural Use-Cases

Use-Case 1: Upload Model to Hub

Workflow (Python SDK):
```python
from huggingface_hub import HfApi, HfFolder
from transformers import AutoModel, AutoTokenizer
import json

# Authenticate
HfFolder.save_token("your_hf_token")
api = HfApi()

# Create repository
repo_id = "username/my-model"
api.create_repo(
    repo_id=repo_id,
    repo_type="model",
    private=False,
    exist_ok=True
)

# Upload model files
model = AutoModel.from_pretrained("bert-base-uncased")
model.push_to_hub(repo_id)

# Upload tokenizer
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
tokenizer.push_to_hub(repo_id)

# Upload model card
model_card = """---
language: en
license: mit
tags:
  - bert
---

# My BERT Model

This is my fine-tuned BERT model for [your task].
"""

api.upload_file(
    path_or_fileobj=model_card.encode(),
    path_in_repo="README.md",
    repo_id=repo_id,
    repo_type="model"
)

print(f"Model uploaded to https://huggingface.co/{repo_id}")

```

1.6.1 Use-Case 2: Download and Use Model

Workflow (Python):

```

from transformers import AutoTokenizer, AutoModel
import torch

# Download model and tokenizer
model_id = "username/my-model"
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModel.from_pretrained(model_id)

# Use for inference
text = "This is a sample text."
inputs = tokenizer(text, return_tensors="pt")
outputs = model(**inputs)

print(outputs.last_hidden_state.shape)

```

1.6.2 Use-Case 3: Model Card Creation

Step-by-Step:

```

from huggingface_hub import ModelCard, ModelCardData
from datetime import datetime

# Create model card metadata
card_data = ModelCardData(
    language="en",
    license="mit",
    library_name="transformers",
    tags=["text-classification", "bert"],
    model_name="BERT Classifier",
    model_id="username/bert-classifier"
)

# Create model card
card = ModelCard.from_template(
    card_data,
    model_id="username/bert-classifier",
    model_description="Fine-tuned BERT for sentiment analysis",
    intended_use="Sentiment analysis on product reviews",
    limitations="English text only, trained on product reviews",
    how_to_use="""
    ```python
 from transformers import pipeline
 classifier = pipeline("text-classification", model="username/bert-classifier")
 result = classifier("Great product!")
 print(result)
    ```
    """
)

# Save and upload
card.save("README.md")

```

1.7 Examples

1.7.1 Example 1: Hub API REST Requests

List Models:

```
curl "https://huggingface.co/api/models?search=bert&sort=likes&direction=-1&limit=10"
```

Real Response (JSON):

```
[
  {
    "id": "bert-base-uncased",
    "author": "google",
    "modelId": "google/bert-base-uncased",
    "private": false,
    "pipeline_tag": "fill-mask",
    "tags": ["transformers", "tensorflow", "pytorch"],
    "lastModified": "2023-12-04T10:30:00.000Z",
    "siblings": [
      {"rfilename": "config.json"},
      {"rfilename": "pytorch_model.bin"},
      {"rfilename": "tokenizer.json"}
    ],
    "downloads": 50000000,
    "likes": 15000
  }
]
```

Download File:

```
curl "https://huggingface.co/google/bert-base-uncased/resolve/main/config.json" \
-o config.json
```

1.7.2 Example 2: Python API Usage

```
from huggingface_hub import hf_hub_download, model_info, list_models

# Get model info
info = model_info("google/bert-base-uncased")
print(f"Model: {info.modelId}")
print(f"Downloads: {info.downloads}")
print(f"Likes: {info.likes}")
print(f"Tags: {info.tags}")

# Download specific file
model_file = hf_hub_download(
    repo_id="google/bert-base-uncased",
    filename="config.json",
    cache_dir="/tmp/models"
)

# List models by task
models = list_models(
    task="text-classification",
    sort="downloads",
    direction=-1
)

for model in models[:5]:
    print(f"{model.modelId} - {model.downloads} downloads")
```

1.7.3 Example 3: Complete Upload Workflow

```

from huggingface_hub import HfApi, create_repo, upload_folder
from transformers import AutoModel, AutoTokenizer
import os

def upload_model_to_hub(
    local_model_path: str,
    repo_name: str,
    private: bool = False
):
    """Upload Local model to Hugging Face Hub"""

    api = HfApi()

    # Create repository
    repo_url = create_repo(
        repo_id=repo_name,
        repo_type="model",
        private=private,
        exist_ok=True
    )

    # Prepare model card
    model_card = """---
language: en
license: mit
---

# My Model

This is my fine-tuned model.
"""

    # Write model card
    with open(os.path.join(local_model_path, "README.md"), "w") as f:
        f.write(model_card)

    # Upload entire folder
    api.upload_folder(
        folder_path=local_model_path,
        repo_id=repo_name,
        repo_type="model"
    )

    print(f"Model uploaded to {repo_url}")
    return repo_url

# Usage
upload_model_to_hub(
    "/path/to/my/model",
    "username/my-awesome-model",
    private=False
)

```

1.8 Safetensors Format

Modern format for storing model weights (replaces pickle):

Advantages: - Secure (no code execution) - Fast loading - Smaller file size - Language-agnostic

Usage:

```
import torch
from safetensors.torch import load_file, save_file

# Save
state_dict = model.state_dict()
save_file(state_dict, "model.safetensors")

# Load
state_dict = load_file("model.safetensors")
model.load_state_dict(state_dict)
```

1.9 Tools & Ecosystem

Tool	Description	Homepage
huggingface_hub	Python SDK	https://github.com/huggingface/huggingface_hub
Transformers	Model library	https://huggingface.co/docs/transformers
Diffusers	Image generation	https://huggingface.co/docs/diffusers
Datasets	Dataset library	https://huggingface.co/docs/datasets
Spaces	Demo hosting	https://huggingface.co/spaces
Gradio	Interface builder	https://www.gradio.app/

Navigation: [Back to Index](#) | [Previous: MLflow](#) | [Next: gRPC for ML Serving](#)